CallGuard — Guida completa allo sviluppo

Filosofia: come lavoriamo io e te

Io (Claude) scrivo tutto il codice — backend, frontend, Swift. Tu non devi inventare nulla.

Tu fai tre cose:

1. Copi il codice che ti do e lo incolli dove ti dico
2. Esegui i comandi nel Terminale che ti indico
3. Mi dici gli errori che vedi — io li risolvo

Se qualcosa non funziona, mi incolli l'errore e io ti guido. Non serve capire il codice per farlo funzionare — serve seguire le istruzioni con precisione.

Cosa devi installare (una tantum, ~30 minuti)

Apri il Terminale del Mac e esegui questi comandi uno alla volta:

1. Homebrew (gestore pacchetti per Mac)

```bash
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/i
```

2. Node.js (per il backend)

```bash
brew install node
```

Verifica: `node --version` deve mostrare v20 o superiore.

3. Git (per salvare il codice)

```bash
brew install git
```

4. Xcode

Lo hai già — assicurati di avere anche i Command Line Tools:

```bash
```

```
xcode-select --install
```

5. Account gratuiti da creare

- **Railway** (https://railway.app) — per il backend, piano gratis
- **Vercel** (https://vercel.com) — per la web app, piano gratis
- **GitHub** (https://github.com) — per salvare il codice

Fase 1 — Backend + Web App (giorni 1-4)

Cosa costruiamo

Un server che espone API REST e un sito web dove chiunque può:

- Cercare un numero di telefono e vedere se è spam
- Segnalare un numero come spam/truffa/telemarketing
- Vedere i numeri più segnalati

Stack semplificato per MVP

Cosa	Tecnologia	Perché
Backend	Node.js + Express	Il più documentato, facile da debuggare
Database	SQLite	Un singolo file, zero configurazione
Web app	React + Vite	Veloce da creare, deploy gratis su Vercel
Deploy backend	Railway	Collega GitHub e deploya automaticamente

Giorno 1: Setup progetto + database

Obiettivo: avere il server che parte in locale.

Tu fai:

1. Crei una cartella `callguard` sul Desktop
2. Apri Terminale, vai nella cartella
3. Incolli i comandi che ti do per inizializzare il progetto
4. Incolli i file che ti scrivo
5. Esegui `npm start` — il server parte

Io faccio:

- Ti do il codice completo per: server Express, schema SQLite, prima API `/health`
- Ti spiego dove mettere ogni file

Struttura cartelle:

```
callguard/
├── backend/
|   ├── src/
|   |   ├── index.js          ← server principale
|   |   ├── database.js       ← setup SQLite
|   |   ├── routes/
|   |   |   ├── lookup.js     ← GET /api/lookup/:number
|   |   |   ├── report.js     ← POST /api/report
|   |   |   └── feed.js       ← GET /api/feed
|   |   └── services/
|   |       └── scoring.js    ← algoritmo spam score
|   ├── package.json
|   └── data/
|       └── callguard.db      ← database (generato automaticamente)
|
├── webapp/
|   ├── src/
|   |   ├── App.jsx
|   |   ├── pages/
|   |   |   ├── Search.jsx    ← cerca un numero
|   |   |   ├── Report.jsx    ← segnala un numero
|   |   |   └── Feed.jsx      ← numeri più segnalati
|   |   └── components/
|   |       └── SpamBadge.jsx  ← badge visivo dello score
|   ├── package.json
|   └── vite.config.js
|
└── ios/
    └── CallGuard/             ← progetto Xcode (fase 2)
```

Giorno 2-3: API complete + score engine

Obiettivo: tutte le API funzionano, puoi cercare e segnalare numeri.

Io ti do il codice per:

- `GET /api/lookup/:number` — cerca un numero, risponde con spam score
- `POST /api/report` — riceve una segnalazione
- `GET /api/feed` — ultimi numeri segnalati

- Score engine che calcola lo spam score
- Seed data: 50 numeri spam noti precaricati per testare

Tu testi con il browser: vai su `http://localhost:3000/api/lookup/+393331234567`

Giorno 4: Web app React + deploy

Obiettivo: sito online, utilizzabile da chiunque.

Io ti do il codice per:

- Una pagina di ricerca con barra di input
- Una pagina per segnalare con form
- Un feed dei numeri più segnalati
- UI pulita e mobile-friendly

Tu fai:

1. Crei un repository su GitHub
2. Fai push del codice (ti do i comandi esatti)
3. Colleghi Railway → backend online
4. Colleghi Vercel → web app online
5. Hai un sito funzionante con URL pubblico

Fase 2 — App iOS nativa (giorni 5-8)

Cosa costruiamo

Un'app iPhone che:

- Si sincronizza con il backend per scaricare i numeri spam
- Usa CallKit per mostrare "SPAM" quando chiama un numero segnalato
- Permette di segnalare numeri dall'app

Giorno 5: Nuovo progetto Xcode + UI base

Obiettivo: un'app che si apre sul tuo iPhone.

Tu fai:

1. Apri Xcode → New Project → App → SwiftUI
2. Nome: "CallGuard", Organization: il tuo team ID
3. Colleghi il tuo iPhone con il cavo
4. Premi Run → l'app si apre sul telefono

Io ti do:

- Le schermate SwiftUI (cerca, segnala, impostazioni)
- Il codice per chiamare le API del backend

Giorno 6-7: CallKit Call Directory Extension

Obiettivo: quando chiama un numero spam, il telefono mostra "Spam sospetto".

Questa è la parte più delicata. In Xcode:

1. File → New → Target → Call Directory Extension
2. Xcode crea un file `CallDirectoryHandler.swift`
3. Io ti do il codice che: scarica i numeri dal backend, li inserisce nella directory di iOS

Come funziona CallKit (semplificato):

```
Il tuo backend ha una lista di numeri spam
     ↓
L'app scarica la lista ogni 6 ore
     ↓
La passa a iOS tramite Call Directory Extension
     ↓
Quando arriva una chiamata da uno di quei numeri,
iOS mostra automaticamente la label "Spam" o "Telemarketing"
```

Attenzione: l'utente deve attivare manualmente l'extension: Impostazioni → Telefono → Blocco chiamate e identificazione → attiva CallGuard. Questo va spiegato nell'onboarding dell'app.

Giorno 8: Collegamento backend

Obiettivo: l'app scarica i numeri reali dal server.

Io ti do:

- Il codice per chiamare `GET /api/sync/ios` dal backend
- La logica di background refresh
- Il salvataggio locale dei numeri per funzionare offline

Fase 3 — Polish + TestFlight (giorni 9-12)

Giorno 9-10: UI e onboarding

- Schermata di benvenuto che spiega come attivare CallKit

- Animazioni e feedback visivi sulle segnalazioni
- Icona dell'app e splash screen

Giorno 11: Upload su TestFlight

Tu fai:

1. In Xcode: Product → Archive
2. Distribuisci su App Store Connect
3. Crei un gruppo di test su TestFlight
4. Inviti fino a 100 persone via email

Giorno 12: Beta test

- Inviti 10-20 amici/familiari
- Raccogli feedback
- Io ti aiuto a fixare i bug che trovano

Limiti e aspettative oneste

Cosa funzionerà bene

- Ricerca e segnalazione numeri via web e app
- Identificazione chiamate spam su iPhone via CallKit
- Database che cresce con le segnalazioni della community

Cosa sarà limitato nell'MVP

- **Solo iOS** — niente Android per ora (richiede Kotlin nativo)
- **Sync ogni 6 ore** — non in tempo reale (limite Apple)
- **Niente notifiche push** — richiedono un server APNS, lo aggiungiamo dopo
- **UI semplice** — funzionale ma non da designer professionista
- **Niente account utente** — identifichiamo per device ID, basta per l'MVP
- **SQLite** — regge bene fino a ~100K numeri e qualche centinaio di utenti

Quando servirà aiuto esterno

- Se l'app decolla (>1000 utenti) → serve migrare a PostgreSQL
- Per pubblicare su App Store (non TestFlight) → serve review Apple, privacy policy seria
- Per Android → serve uno sviluppatore Kotlin

- Per il GDPR formale → serve un consulente legale

Costi

Voce	Costo
Apple Developer (lo hai già)	99€/anno
Railway backend (free tier)	0€
Vercel web app (free tier)	0€
Dominio callguard.it (opzionale)	~10€/anno
Totale per l'MVP	**~0€**

Pronto?

Quando vuoi partire, dimmi "Giorno 1" e ti do il primo blocco di codice con le istruzioni passo-passo per il setup del progetto.